DAY 2 · FRIDAY 23 OCTOBER · CAPSTONE



Capstone: Design an Agentic System

Architecture design and technical design — think first, no code

Pavlos Protopapas
Harvard University

THE IDEA

Think and design — do not build

- By **Saturday 9:00**, your team designs an LLM system for a real problem. **No code.**
- You hand in two things: an **architecture design** and a **technical design**.
- On Saturday we review them together, and argue about them.
- Why design and not build? In practice the expensive mistakes happen before the first line of code: the wrong architecture, no way to know if it works, no plan for when it fails.

You will practise

Choosing the **simplest** architecture that works

Turning it into concrete choices: models, data, tools, memory, guardrails

Deciding **how you will know** it works — before building it

Estimating **cost and latency**

Naming how it **fails**

TIMELINE

Twenty-four hours, mostly in the breaks

Fri 9:00	Fri breaks	Fri 17:30	Fri evening	Sat 9:00	Sat 11:20
This brief	**Talk**	**Worked example**	**Write it up**	**Hand in**	**Review**
teams of 4–5, pick a scenario	coffee, lunch, coffee: agree on the problem and the architecture	we design a pisco sour assistant, end to end	fill in the template: 6 slides	one PDF per team	teams present, we discuss

Teams

4–5 people, ideally from **different organisations**. Pick one scenario (next slides) — or bring a problem from your own work, if you can describe it without confidential details. Tell us your team name and scenario at lunch.

THE DELIBERATE TRAP

Climb the ladder only as far as you must

Multi-agent	several agents hand off work
Single agent	the model picks tools in a loop
RAG	retrieve, then answer
Pipeline	fixed steps
Prompt	one call, no data

- Every scenario **looks** like it wants a swarm of agents. Most do not.
- Each rung adds capability — and cost, latency and new ways to fail.
- Go one rung up only when you can say **what breaks** on the rung below.
- We score **judgment**, not the number of agents. A clean single agent with a real evaluation beats an impressive multi-agent system that is slower, costlier and no more correct.

SCENARIOS

Pick one

A · Support triage

A bank, telco or utility receives thousands of customer messages a day. For each, **answer** from the help articles and policies, **route** it to the right team, or **escalate** to a human.

Hard part: knowing what it does not know.

B · Research brief

An analyst asks a question. The system gathers from a document collection plus **one external source** (web or a data API) and writes a **structured, cited** brief.

Hard part: never inventing a source.

C · Operations assistant

An IT or operations team runs multi-step procedures (restart a service, refund an order, close an account) by calling tools, with **human approval** before anything irreversible.

Hard part: an approval gate that cannot be talked around.

D · Your own problem

A problem from your work that an LLM system could help with. Same deliverables. Check with us at lunch that it is the right size for one evening.

Hard part: scoping it down.

DELIVERABLE 1

The architecture design

- **The problem** in one sentence, and who uses the system.
- **The decision**: which rung of the ladder — prompt, pipeline, RAG, single agent, multi-agent.
- **Why not one rung simpler?** What would break.
- **Why not one rung fancier?** What it would cost you.
- **The diagram**: components and how data flows for one real request.
- **Where humans are** in the loop, if anywhere.
- **The failure mode** the architecture is built to guard against.

Good answers sound like

"A single agent with three tools. A fixed pipeline cannot decide when it needs the price list; a second agent would only add a hand-off where context gets lost."

"The one failure we design against: recommending a product that is not in stock. Every product name in an answer must come from the catalog tool."

DELIVERABLE 2

The technical design

Decide	Say concretely	Example
Models	which model for which job, and why (quality, cost, latency)	gpt-4o-mini for the agent; no reasoning model
Knowledge	sources, chunking, embeddings, vector store, top-k	350-character chunks, 256-d embeddings, top 5
Tools	name, inputs, outputs — and why it is a tool, not the LLM	scale_recipe(servings) → exact quantities
Memory and state	what is kept, for how long, where	party size and budget, for this session only
Guardrails	input and output checks, refusals	every product in an answer must exist in the catalog
Evaluation	test cases, metric, pass bar	40 cases; 0 invented products; ≥ 90 % correct
Cost and latency	tokens per request, $ per month, seconds	≈ $0.005 per conversation, < 5 s
Risks	the top three, and what you do about each	prompt injection in product descriptions

HAND-IN

Six slides, one PDF, Saturday 9:00

1	**Team and problem**	team name, members, scenario; the problem in one sentence; users
2	**Architecture decision**	the rung; why not simpler; why not fancier; the failure mode
3	**Architecture diagram**	components and the data flow of one real request
4	**Technical design**	models, knowledge, tools, memory, guardrails
5	**Evaluation plan**	test cases, metric, pass bar — how you will know it works
6	**Cost, latency and risks**	the numbers, and the top three risks

How

Start from the **template** (on the team page).

Save as PDF: **Team_<name>.pdf**.

Send it by **Saturday 9:00** — link on the team page.

Hand-drawn diagrams are fine: take a photo.

ROLES

Who does what in a team of 4–5

Architect

owns the decision on the ladder, and says no to scope creep

Tech lead

models, data, tools: makes the technical design concrete

Evaluator

the test cases and the pass bar — owns "how do we know"

Red-teamer

how it fails: wrong answers, attacks, cost blow-ups

Presenter

five minutes on Saturday, and the first answer to our question

In a team of four, the presenter also plays one of the other roles. Everyone should be able to defend the design.

SATURDAY 11:20

How we review

- Selected teams present: **5 minutes** — problem, decision, diagram, evaluation.
- Then **one hard question** from us, and a discussion with the room. Typical questions:
  - Why is this not just a prompt, or a pipeline?
  - You added a second agent — what does it buy you?
  - What would make your evaluation pass while the system is wrong?
  - Where does it break at 10× the users, or 10× the cost?
  - What happens when a tool returns something malicious?

We look for	
Right-sized	the architecture fits the problem
Clear flow	one request, end to end
Concrete	named models, tools, numbers
Testable	a real evaluation plan
Honest	names its failure modes
Cost-aware	tokens, dollars, seconds

Not scored: number of agents, trendy frameworks, pretty slides.

Tips

- Write the problem in **one sentence** before you draw anything.
- Pick **three real requests** a user would make. Design for those; check the design against each.
- Put **numbers** on it: how many users, how many tokens per request, how many seconds the user waits.
- For every LLM call, ask: could this be **ordinary code**? (Arithmetic, lookups and rules usually should be.)
- Decide what you will **not** build. Say it on the slide.
- At 17:30 you will see a complete example. Steal its structure, not its scenario.